UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

22. Testbenches

UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

22.1 Basic Testbench

UNIVERSITY OF Southampton
School of Electronics and Computer Science

What is a testbench?

- Top level module in design, with no ports
- Used for testing and verification
  - the testbench is not synthesised, used in simulation only
- May contain one or more initial blocks with clocks and input waveforms

Typical testbench

UNIVERSITY OF Southampton
School of Electronics and Computer Science

```
// Tested module, register with parallel load
// and 3-state output:
module regs (input logic clk, ctrl, load, reset, input logic
[7:0] D, output logic [7:0]  Q);

logic [7:0] buf; // buffer memory

always_ff @ (posedge clk)
    if (reset)
            buf <= 8'h00;
    else if (load)
            buf <= D;

// 3-state output
assign Q = (ctrl ? buf:  8'bZZZZ_ZZZZ);

endmodule
```

```
module testbench;

logic[7:0] Q,D;
logic clk,ctrl,load,reset;

// register instance
regs r (.*); //use default size – 8 bits

initial // clock block
begin
    clk = 1'b0;
   forever #10 clk = ~clk;
end;

initial
begin
    D = 8'hAA; // sample data
    ctrl = 1'b1; // drive output
    load = 1'b1; // load D
    reset = 1'b1; // clear register
  #50 reset = 1'b0; // remove reset
  #50 ctrl = 1'b0; // remove ctrl, outputs go hi-z
end; // 2nd initial block

endmodule;
```

Example 2

UNIVERSITY OF Southampton
School of Electronics
er Science

This module contains two instances of module regs with two sets of inputs and outputs and two separate control signals ctrl1 and ctrl2.

Note that the outputs of both registered are wired together to port Q

Also note the use of named port mapping

Write a suitable testbench.

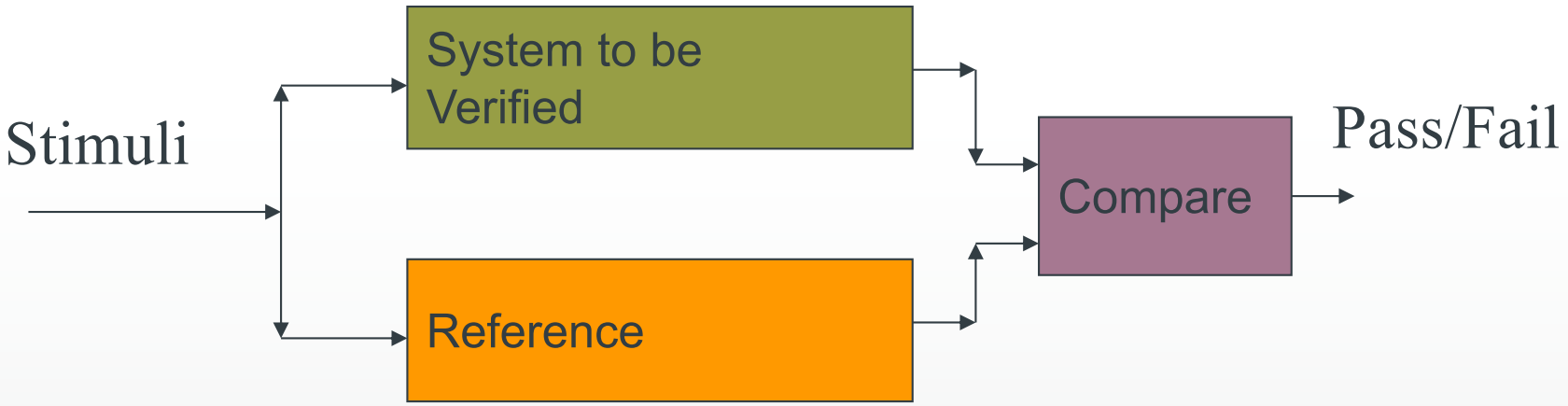
As a conflict will occur if both wired outputs are driven at the same time, the testbench should display a message if at least one of the registers does not have its outputs at the high impedance state at any point of time.

```
module tworegs  #(parameter n=8) (output logic[n-1:0] Q,
input logic[n-1:0] D1,D2, input logic clk, ctrl1, ctrl2, load, reset );

// register instances – nb: outputs might
regs r1 (.Q(Q),.D(D1),.ctrl(ctrl1),.load(load),.reset(reset),.clk(clk));
regs r2 (.Q(Q),.D(D2),.ctrl(ctrl2),.load(load),.reset(reset),.clk(clk));

endmodule
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Checking Responses

- Compare with reference



- Reference could be behavioural (simulation) model

Testbench 2

UNIVERSITY OF Southampton
School of Electronics and Computer Science

```
module testbench;

logic[7:0] Q,D1,D2;
logic clk,ctrl1,ctrl2,load,reset;

// instance of the tworegs module
tworegs r (.*); //use default size – 8 bits

initial // clock block
begin
    clk = 1'b0;
   forever #10 clk = ~clk;
end;

// concurrent assertion to detect if both
// outputs are driven at same time
 assert property (@(posedge clk)
      ~ctrl1 | ~ctrl2 )
      else
      $error("bus conflict at time %0t", $time);

…
```

```
 …

initial
begin

  D1 = 8'hAA; // data
  D2 = 8'11;
  ctrl1 = 1'b1; // drive output1
  ctrl2 = 1'b0;// disconnect output 2
  load = 1'b1; // load D1,D2
  reset = 1'b1; // clear registers

#50 reset = 1'b0; // remove reset
#50 ctrl1 = 1'b0; // remove ctrl1, outputs go hi-z

//assertion above should now fail:
#50 ctrl11 = 1'b1; ctrl2 = 1'b1; // both registers driven


end;
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Synchronising Signals

- Suppose we want to change a signal 10 ns after a rising clock edge:

```
initial // or always?
   begin
   @(posedge clock);
   // wait for clock to change to 1
   #10ns a = a + 1;
   end
```

- It *might* be better to use a nonblocking assignment!

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Randomly-Timed Inputs

- "Real World" inputs are random w.r.t. clock
- Need random number generator
- Negative exponential function is a good approximation to random arrival times:

#($dist_exponential(seed, 10000));

- Need to specify time units

```
timeunit 1ns; timeprecision 100ps;
```

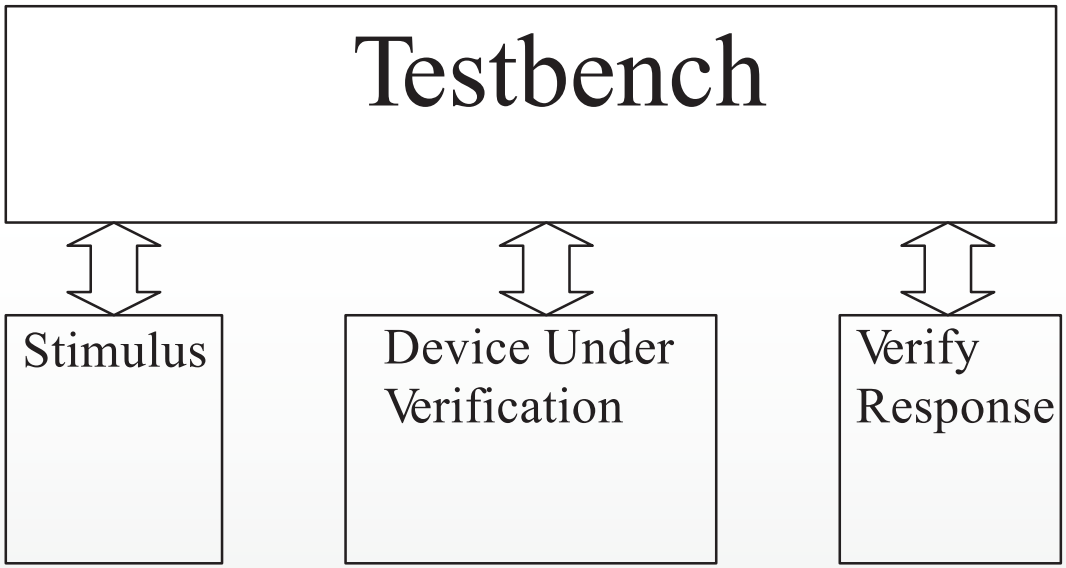
UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

22.2 Testbench Example

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Modular testbench example

- Simple example to illustrate a layered testbench



- Verify a sequential Booth multiplier (DUV)

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Top Level

```
module testbooth;

parameter N = 16;

logic signed [N-1:0] ain, bin;
logic signed [2*N-1:0] qout;
logic clk, load, n_reset;
logic ready;

clock_gen #(10.0, 10.0) c0 (.*);
stimulus s0 (.*);
booth #(N, N, 2*N) b0 (.*);  //DUV
verify v0 (.*);

endmodule
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Booth Multiplier

```
module booth #(parameter AL = 8, BL = 8, QL = AL+BL)
   (output logic [QL-1:0] qout, output logic ready,
    input logic [AL-1:0]ain, input logic [BL-1:0] bin,
    input logic clk, load, n_reset);

   .
   .
   .

endmodule
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Clock generator

```
module clock_gen
 #(parameter ClockFreq_MHz = 100.0,  ResetWidth=10.0)
  (output logic clk, n_reset);
timeunit 1ns;
timeprecision 100ps;

parameter ClockHigh = (500.0)/ClockFreq_MHz;

initial
  begin
  n_reset = '1;
  clk = '0;
  #ResetWidth n_reset = '0;
  #ResetWidth n_reset = '1;
  forever #ClockHigh clk = ~clk;
 end

endmodule
```

Note that this is a module

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Stimulus

```
module stimulus #(parameter NA = 16, NB = 16)
   (output logic signed [NA-1:0] ain,
    output logic signed [NB-1:0] bin,
    output logic load);

initial begin
 #30ns ain = -10; bin = 10;
 #10ns load = '1;
 #10ns load = '0;
 end


 endmodule
```

Verify

UNIVERSITY OF Southampton
School of Electronics and Computer Science

```
module verify #(parameter AL = 8, BL = 8, QL = AL+BL)
(input logic signed [QL-1:0] qout,
input logic ready,  input logic signed [AL-1:0]ain,
input logic signed [BL-1:0] bin, input logic clk, load);

always @(posedge clk)
 $display("%d * %d gives %d", ain, bin, qout);

endmodule
```

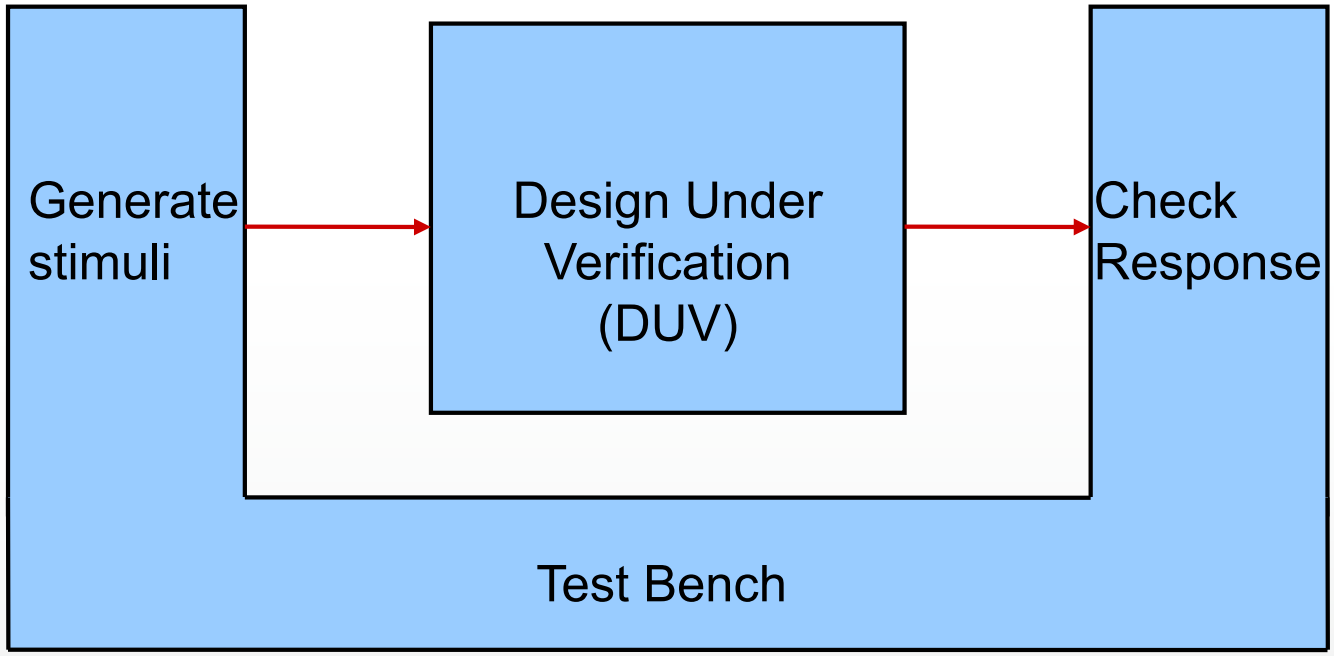
- Could also check that ready flag is asserted at the right time (after AL cycles)
- SystemVerilog also has assertions
  - We can assert that a property will be true in a particular cycle
  - Syntax is different to synthesisable code, but time dependencies can be concisely expressed

UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

22.2 Testbench Strategies

UNIVERSITY OF Southampton
School of Electronics and Computer Science

The Basic Testbench



UNIVERSITY OF Southampton
School of Electronics and Computer Science

Verification Plan

- As the design develops, do you want to keep testing the same things?
  - Check to see that the design has not been broken (regression testing)
  - Check new things
- One simulation run? Or a number of smaller simulations?
- Need to write a verification plan – what to verify, when, how

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Divide and Conquer

- Your design isn't one huge monolithic block
- Why should the testbench be like that?
- Issues:
  - Verification plan probably requires many different tests
  - Need to test several versions of the design
  - Create environment around the design
- Objectives:
  - Flexibility
  - Low-effort maintenance and enhancement
  - Reliability
  - Re-use

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Monolithic Testbenches are Inflexible

- Consider what would need to be changed to run different test cases
  - *test case* = group of tests required by one aspect of the verification plan, run as one simulation
- Stimulus generator and file output writers would need to change
- Top-level testbench then changes to accommodate them

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Package Useful Functionality

- High-level component models for other parts of the system
- Stimulus generator
- Output checker
- Logging and monitoring
- Utility functions (application-specific calculations)

UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

22.3 Verification Methodologies

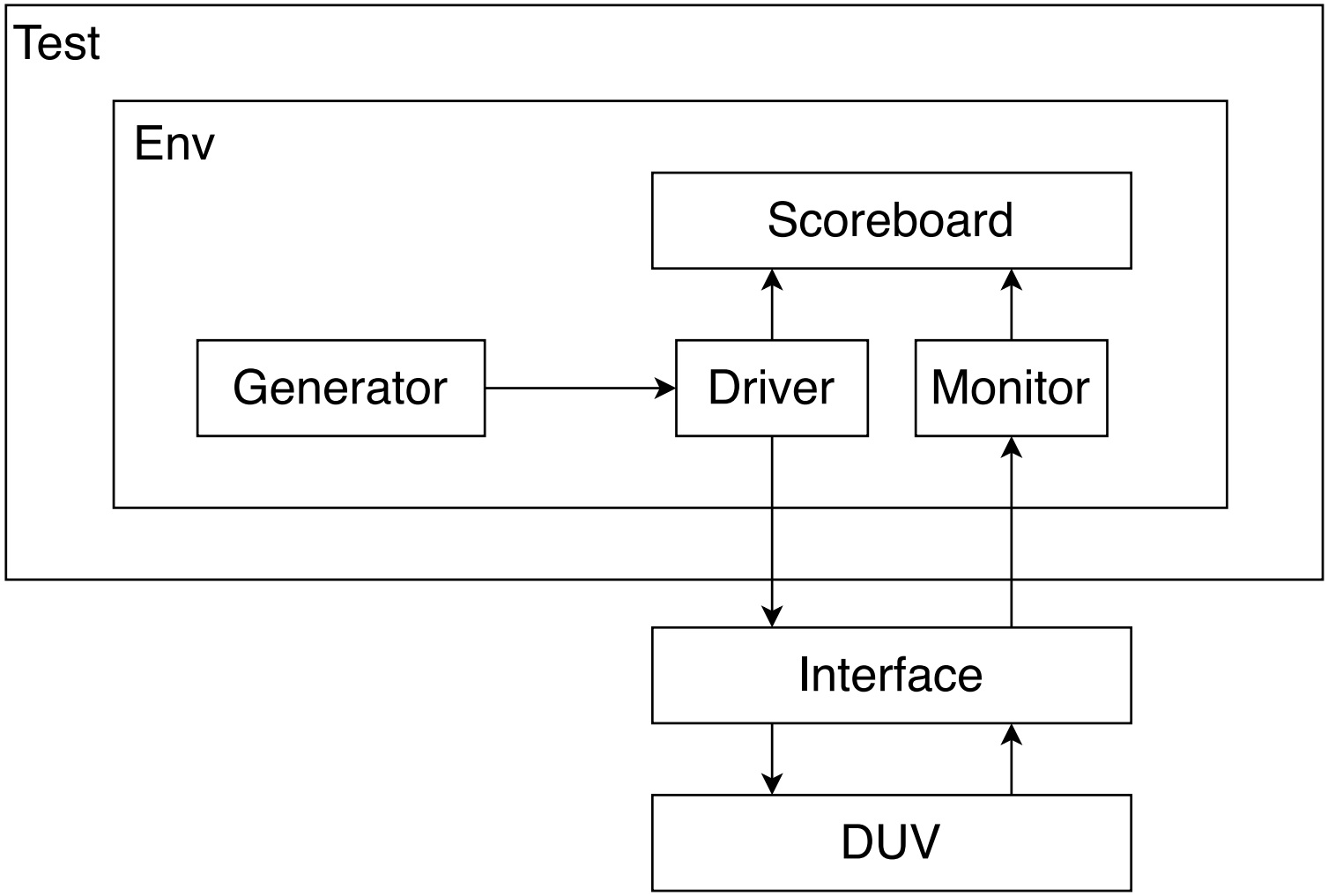
UNIVERSITY OF Southampton
School of Electronics and Computer Science

Standards

- Universal Verification Methodology (UVM) is an industry standard for verifying designs.
  - SystemVerilog class library for simulation
- cocotb is an open source approach backed by Synopsys, Cadence, Siemens, Aldec
  - Python class library
- Similar objectives, but very complex

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Testbench structure



UNIVERSITY OF Southampton
School of Electronics and Computer Science

SystemVerilog constructs

- class – Test, Env, Scoreboard, Driver, Monitor are defined as SystemVerilog classes (encapsulation)
- interface – used to group signals
  - virtual interface – group signals between objects and modules